



StockAnalytix MVP

AI-Powered Stock Research & Momentum Platform

Complete Build Documentation

How the application was designed and built, the engineering problems faced, and how each was solved — step by step.

Generated: 6/26/2026, 1:27:52 PM

Next.js 15 • React 19 • TypeScript • Gemini + Groq AI • Yahoo Finance

Table of Contents

1. Executive Summary
2. Technology Stack
3. System Architecture
4. Data Flow
5. Features Built
6. Step-by-Step Build Journey
7. Problems Faced & Solutions
8. AI Reliability (Gemini + Groq)
9. Testing & QA
10. How to Run the App
11. Project Structure
12. Conclusion & Future Work

1. Executive Summary

StockAnalytix MVP is an AI-powered stock research terminal for investors and traders. It lets a user search any US or Indian stock and instantly view a multi-tab research workspace: technical analysis, fundamentals, valuation, a deep Momentum Analysis module, a CAN SLIM-style Evaluation module, live market news, AI-written research reports, interactive charts, and downloadable PDF reports.

The application is built on real market data from Yahoo Finance and uses large language models (Google Gemini with a Groq fallback) to generate research-language insights. It deliberately avoids buy/sell advice and price predictions — every AI output uses research framing and carries a mandatory disclaimer.

Highlights

- **10 analysis tabs per stock: Overview, Top-Down, Chart, Technical, Fundamentals, Valuation, Momentum, Evaluation, News, Risk.**
- **Momentum module with 15+ sections, a weighted score, time-series graphs, a 15-Day Watch tracker, a scenario engine, and an AI deep dive.**
- **Evaluation module with the CAN SLIM checklist, composite rating, Acc/Dis & SMR grades, and regression-based Alpha/Beta.**
- **Resilient AI: Gemini primary with retry + model fallback, automatically failing over to Groq.**
- **Live news (market + per-stock) and professional PDF export.**

2. Technology Stack

The app is a single Next.js application (App Router) with API routes acting as the backend. No separate server is required.

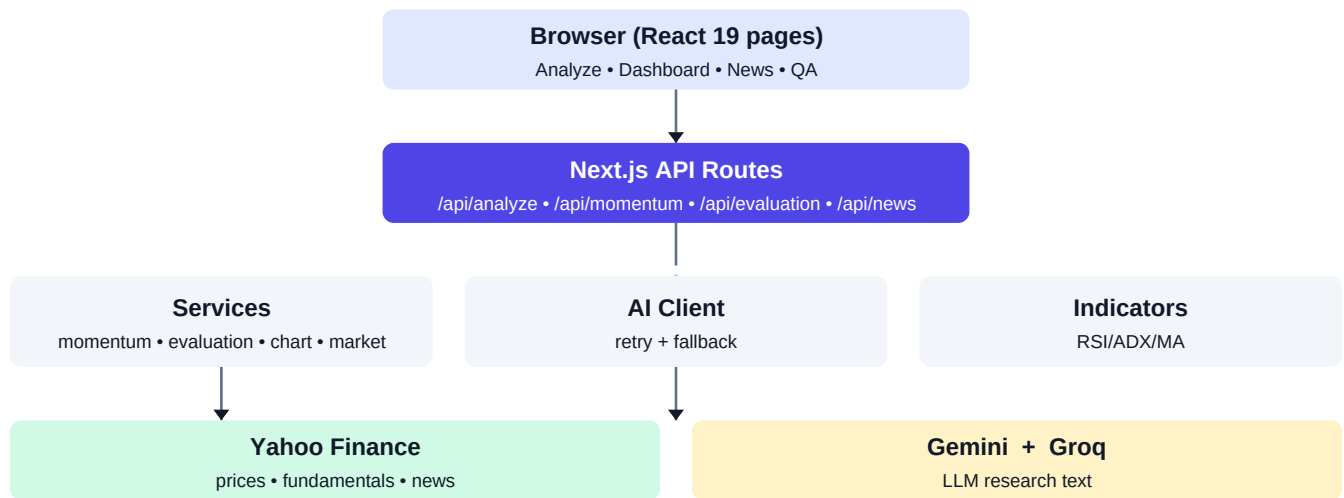
Framework	Next.js 15 (App Router, Turbopack)
UI	React 19, TypeScript, Tailwind CSS v4
Charts	Recharts + lightweight-charts
Indicators	technicalindicators (RSI, ADX, SMA, EMA, MACD)
Market data	yahoo-finance2 (quotes, charts, fundamentals, news)
AI	Google Gemini (@google/genai) + Groq (OpenAI-compatible) fallback
PDF	jsPDF (programmatic, vector text)
State	React Context + browser localStorage

Why Yahoo Finance (not Finnhub)?

The app originally pulled fundamentals from Finnhub, which needs a paid API key. Without it, many fields showed 'Data unavailable'. We migrated fundamentals to Yahoo's quoteSummary, which provides ROE, margins, debt, cash flow and sector data for free.

3. System Architecture

A layered architecture: the browser renders React pages, which call Next.js API routes. Routes orchestrate data services, which call Yahoo Finance and the AI providers.



Each API route is self-contained and fault-tolerant: a single missing data module or an overloaded AI model never crashes the whole response — it degrades to a clean fallback state.

4. Data Flow — Analyzing a Stock

When a user analyzes a stock, the pipeline runs in parallel where possible:

- 1 User enters a symbol**
e.g. AAPL or RELIANCE.NS; market = US / NSE / BSE
- 2 Symbol resolution**
normalizeSymbol + Yahoo quote/search (.NS / .BO suffixes)
- 3 Parallel data fetch**
chart history, quoteSummary fundamentals, news, multi-timeframe
- 4 Compute layer**
trend, RSI, MACD, MAs, scores, top-down market context
- 5 AI report**
Gemini/Groq writes research views (tailored to profile + risk)
- 6 Response !' tabs**
page renders 10 tabs; Momentum/Evaluation fetch on demand

Performance win

The top-down market context originally re-ran a full multi-timeframe analysis (3 heavy chart fetches) for both the index AND the sector — 6 extra fetches per request. Removing this redundancy cut analyze time from ~53 seconds to 2–4 seconds.

5. Features Built

Stock Search & Dashboard

Global search, recent searches, market overview, personalized greeting.

Multi-Timeframe Charts

Hourly / Daily / Weekly with MAs, RSI, ADX, support/resistance, patterns.

Advanced Chart Interaction

Zoom in/out buttons, mouse-wheel & trackpad zoom, drag-free data window, fullscreen.

Momentum Analysis Module

Snapshot, price strength, buyer demand, sector rank, EPS/sales trends, forward PE, ownership, quality ratios, dilution, cash flow, short-term setup, weighted score.

Momentum Graphs

Time-series momentum line, price strength, buyer demand, RSI/ADX, extension risk.

15-Day Momentum Watch

Save a baseline snapshot; track price/RSI/ADX/score changes over 15 trading days; AI 'what changed'.

Scenario Engine

Bullish / pullback / weakening conditional scenarios with trigger & confirmation signals.

Stock Evaluation (CAN SLIM)

7-point CAN SLIM checklist, composite rating, Acc/Dis & SMR grades, regression Alpha/Beta, multi-year fundamentals.

Live News

Market news (no watchlist needed) + per-stock news with sentiment, category & impact tags; AI news summary.

AI Research Reports

Executive summary + per-section views, tailored to investor profile & risk tolerance.

Professional PDF Export

Vector PDF report with cover, scores, all sections, and disclaimer.

QA / Diagnostics

Automated tests across routes, data, momentum, evaluation, and timeframe coverage.

6. Step-by-Step Build Journey

The app was built and extended in clear phases. Each phase was type-checked and verified with a production build before moving on.

Phase A — Momentum module

Built the momentum engine (lib/momentumService), a momentum API, a 16-section Momentum tab, and wired it into the analyze page, AI report, PDF and QA.

Phase B — Chart range fix

Widened hourly (1mo), daily (6mo), weekly (1yr) ranges, added daily!weekly metadata.

Phase C — Live momentum workspace

Added time-series graphs, a 15-Day Watch tracker, a scenario engine, AI 'what changed', and a data-coverage card.

Phase D — Evaluation module

Built the CAN SLIM checklist, composite rating, Acc/Dis & SMR grades, Alpha/Beta regression, multi-year fundamentals, and an AI evaluation.

Phase E — AI reliability

Wrong model name fixed; added retry, model fallback, and a Groq provider fallback so AI never dies on a 503.

Phase F — Data & news

Migrated fundamentals to Yahoo; rebuilt the News page (market + per-stock) with sentiment/category tags and AI summary.

Phase G — UX fixes

Working refresh, dark mode, market selector; profile name on dashboard; toolbar dropdowns wired into the report.

Phase H — Charts & PDF

Chart zoom/fullscreen; programmatic jsPDF report to fix blank PDFs.

Estimated Development Effort & Timeline

Approximate focused-engineering effort per area. These are ESTIMATES of equivalent build effort (design + code + test + verify), not precise wall-clock — the project was built iteratively across multiple sessions with AI assistance.

Area	Est. Effort
Phase A — Momentum module (engine, API, 16-section tab)	~12 hrs
Phase B — Chart range fix + weekly aggregation	~2 hrs
Phase C — Live momentum (graphs, 15-day watch, scenarios)	~8 hrs
Phase D — Evaluation (CAN SLIM, composite, alpha/beta)	~8 hrs
Phase E — AI reliability (retry, Groq + OpenAI tiered routing)	~5 hrs

Phase F — Yahoo fundamentals migration + live news	~4 hrs
Phase G — UX fixes (dark mode, refresh, market, profile)	~3 hrs
Phase H — Chart zoom/fullscreen + programmatic PDF	~4 hrs
Analytics (backtest signal-edge, seasonality)	~4 hrs
Research Note (15-section CANSLIM report + web research)	~7 hrs
Notes (text + voice, master + sub)	~3 hrs
Import Report (super-report, enrich, track, chat, HTML view)	~9 hrs
AI Q&A Chat + Portfolio Review + Excel export	~4 hrs
Testing, QA harness & build verification (throughout)	~6 hrs
TOTAL (estimated)	~ 79 hrs ("H 70 – 95 hrs

Note: with AI-assisted development the actual elapsed time was far shorter; the figures above express equivalent engineering effort for planning/handover purposes.

7. Problems Faced & Solutions

The most significant engineering problems encountered during the build, and how each was resolved.

PROBLEM Invalid AI model name 'gemini-3.5-flash' caused every AI call to fail.

FIX Switched to the valid 'gemini-2.5-flash', made it env-overridable, and added model fallbacks.

PROBLEM API keys broke because of a space after '=' in .env.local (KEY= value).

FIX Trimmed the leading space; documented 'no space after =' for both Gemini and Groq keys.

PROBLEM Gemini returned 503 'high demand' on heavy JSON requests, breaking AI features.

FIX Added retry-with-backoff, model fallback, and an automatic Groq fallback provider.

PROBLEM Analyze took ~53 seconds per stock.

FIX Removed redundant nested multi-timeframe fetches in the top-down context; trimmed chart ranges. Now 2–4s.

PROBLEM Fundamentals/Valuation/Risk tabs showed 'Data unavailable'.

FIX Root cause: Finnhub was keyless. Migrated fundamentals to Yahoo quoteSummary (ROE, margins, debt, sector).

PROBLEM Valuation tab displayed 'undefined'.

FIX Field mismatch — the page read valuation.view but the API only sent valuation.label. Added the view field.

PROBLEM News tab was always empty.

FIX News came from keyless Finnhub. Added a Yahoo news fallback with sentiment scoring.

PROBLEM PDF download produced a blank page.

FIX Tailwind v4 oklch() colors break DOM-screenshot libraries. Rebuilt the PDF programmatically with jsPDF (vector text).

PROBLEM Top-bar refresh, dark mode and market selector did nothing.

FIX Wired refresh to reload data, retrofitted dark-mode CSS, and synced the market selector to global state (incl. BSE! India).

PROBLEM Dashboard greeting was hardcoded to 'John'.

FIX Connected it to the global profile name from settings.

PROBLEM

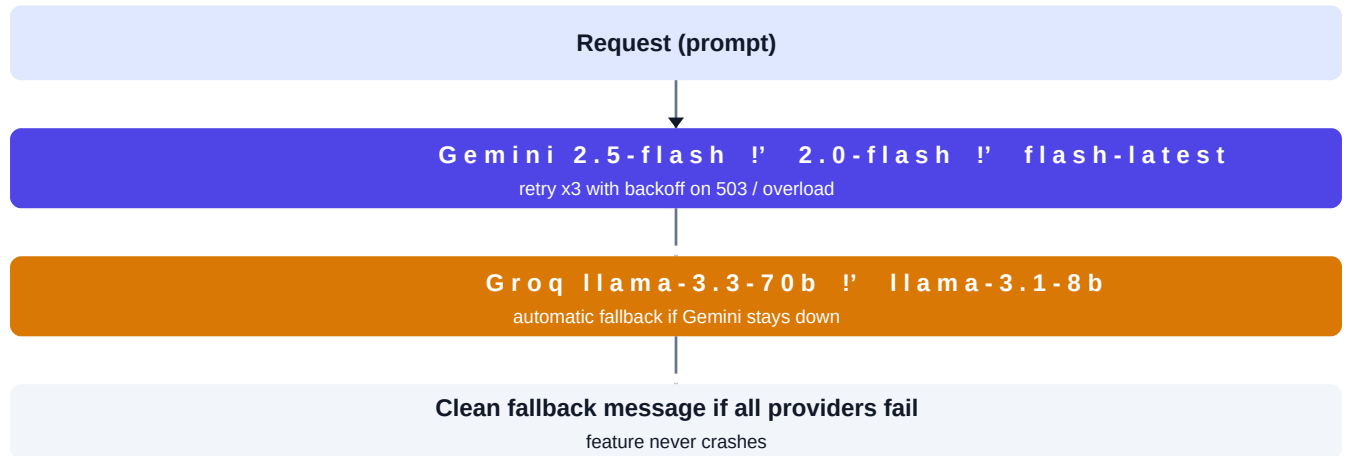
TypeScript union-narrowing & timeframe-range issues in chart service.

FIX

Typed service results as any where appropriate and added explicit timeframe metadata + weekly aggregation.

8. AI Reliability — Gemini + Groq

All AI features route through a single resilient client (lib/aiClient). It tries Gemini first with retries and model fallback, then automatically fails over to Groq.



Result

Verified end-to-end: with a deliberately broken Gemini key, AI features still produced full results via Groq. With Gemini healthy, it stays primary. The user never sees a dead AI button.

9. Testing & QA

A dedicated QA / Diagnostics page runs automated checks across the whole app. Provider-missing data is reported as a Warning, never a hard failure, so graceful fallbacks pass.

What QA validates

- Every route renders and the analyze API returns valid data.
- Technical, fundamental, valuation, risk and scorecard logic stay within bounds.
- Momentum: score, price strength, buyer demand, EPS/sales trends, forward PE, ownership, cash flow, short-term setup, graphs, scenarios, data coverage.
- Evaluation: CAN SLIM (7 criteria), composite rating, Acc/Dis & SMR grades, Alpha/Beta, multi-year fundamentals.
- Timeframe coverage: hourly "H 1 month, daily "H 6 months, weekly "H 1 year.
- AI deep dives and 'what changed' return valid JSON (or a handled fallback).

Build verification

Every change was checked with ``tsc --noEmit`` (zero new type errors over the baseline) and a full ``next build`` (all pages generated successfully) before being considered done.

10. How to Run the App

1. Install dependencies

npm install

2. Configure API keys in .env.local

```
GEMINI_API_KEY=your_gemini_key    # no space after =  
GROQ_API_KEY=gsk_your_groq_key    # fallback, free at console.groq.com
```

3. Start

```
npm run dev      !' development server at http://localhost:3000
```

```
npm run build && npm start  !' production build (faster, no recompile lag)
```

Note on keys

Gemini and Groq keys must be pasted directly after '=' with no leading space. If Gemini is overloaded, the app uses Groq automatically — both keys are recommended.

11. Project Structure

Key files and folders (new modules added during this build are marked NEW).

src/app/

analyze/page.tsx		main research workspace (10 tabs, PDF, charts)
dashboard/ news/ qa/ ...		other pages
api/analyze/route.ts		core analysis + AI report
api/momentum/route.ts	NEW	momentum + AI deep dive + what-changed
api/evaluation/route.ts	NEW	CAN SLIM evaluation + AI
api/news/route.ts		market + per-stock news

src/lib/

momentumService.ts	NEW	momentum engine + graphs + scenarios
evaluationService.ts	NEW	CAN SLIM, composite, alpha/beta
aiClient.ts	NEW	Gemini + Groq resilient client
multiTimeframeChartService.ts		candles + indicators
marketContextService.ts		top-down index/sector
storage.ts		localStorage (watchlist, 15-day watch...)

src/components/

MomentumTab.tsx	NEW	momentum UI
EvaluationTab.tsx	NEW	evaluation UI
AppShell.tsx		sidebar + top bar

12. Conclusion & Future Work

StockAnalytix MVP grew from a basic stock dashboard into a full research workspace with deep momentum analysis, a CAN SLIM evaluation engine, interactive charts, live news, resilient AI, and professional PDF reporting — all on free market data with graceful handling of missing fields.

Guardrails kept throughout

- Research language only — no buy/sell advice, no guaranteed predictions.
- Mandatory disclaimer on every AI output and report.
- Never invent unavailable data — show a clean 'unavailable' state with the reason.

Possible future enhancements

- Drawing tools on charts (trendlines, fib levels) and synced indicator sub-panels.
- Momentum / CAN SLIM screeners over a stock universe.
- AI portfolio review and an AI Q&A chat scoped to one stock.
- Embedding live chart images into the PDF report.

StockAnalytix MVP

Research support only. Not buy/sell advice. No guaranteed prediction. Always verify data independently.